

DMD Controller

DMD Controller — Manuale di installazione

Servizio che pilota il pannello LED 256×64 (chip FM6373) da Raspberry Pi, si presenta a Batocera come un **ZeDMD-WiFi** e offre un'interfaccia web di controllo.

Repository: <https://github.com/kWGillo/zedmd-pi> Versione documentata: **1.8**

Dalla versione 1.5 il software si installa e si aggiorna direttamente da GitHub: non serve più trasferire archivi a mano.

1. Prerequisiti

Il Raspberry deve essere già preparato secondo la procedura hardware del progetto (manuale *DMD Pi — procedura di preparazione*):

- audio integrato disattivato (`dtparam=audio=off` in `/boot/firmware/config.txt`)
- `isolcpus=3` in `/boot/firmware/cmdline.txt` sui modelli quad-core
- libreria `rpi-rgb-led-matrix_pwm_experiment` clonata e compilata nella home
- pannelli cablati e già verificati con la demo del quadrato rotante

Serve inoltre l'accesso SSH al Raspberry. In questo manuale l'utente è `gillo` e l'host è `dmdpi.local`.

2. Installazione da GitHub

Tutto si fa via SSH sul Raspberry. Non serve più il Mac come intermediario.

```
ssh gillo@dmdpi.local

sudo apt update
sudo apt install -y git

git clone https://github.com/kWGillo/zedmd-pi.git ~/dmd
cd ~/dmd
chmod +x install.sh update.sh setup_share.sh
sudo ./install.sh
```

Lo script esegue in sequenza:

1. installazione dei pacchetti di sistema (Flask, NumPy, Pillow, Cython, font, ffmpeg, Samba)
2. **compilazione dei binding Python** della libreria matrice
3. copia dei file in `/opt/dmd`
4. creazione della configurazione in `/etc/dmd/config.json`
5. creazione della libreria media e della condivisione SMB
6. registrazione del servizio systemd `dmd`

Il passo 2 è lento. Compila da sorgente l'intera libreria C++: da qualche minuto su Pi 4 fino a oltre dieci minuti su Pi Zero 2 W. È normale, non interrompere. Il cursore che gira indica che sta lavorando.

Se la libreria matrice non si trova nella home dell'utente:

```
sudo MATRIX_DIR=/percorso/della/libreria ./install.sh
```

Se la cartella `~/dmd` esiste già

Da un'installazione precedente fatta con il pacchetto `.tar.gz`:

```
mv ~/dmd ~/dmd-vecchia
git clone https://github.com/kWGillo/zedmd-pi.git ~/dmd
cd ~/dmd && sudo ./update.sh
```

La configurazione in `/etc/dmd/config.json` non viene toccata: le impostazioni esistenti restano.

3. Configurazione prima del primo avvio

Apri `/etc/dmd/config.json` e verifica due valori:

Chiave	Valore
--------	--------

panel.slowdown

1 Pi Zero W · 3 Pi Zero 2 W e Pi 3 · 5 Pi 4

panel.chain

numero di pannelli collegati in cascata

Le altre chiavi rilevanti, già corrette di default:

Chiave	Valore	Significato
web.port	8080	interfaccia web
zedmd.http_port	80	handshake ZeDMD, non modificare
zedmd.stream_port	3333	frame DMD (TCP e UDP)
zedmd.grace_seconds	60	quanto ZeDMD trattiene il display dopo l'ultimo frame
zedmd.client_timeout	10	silenzio oltre il quale il client è considerato caduto
ota.repo	kwGillo/zedmd-pi	repository per l'aggiornamento via rete

4. Avvio

```
sudo systemctl start dmd
journalctl -u dmd -f
```

Nel log devono comparire le righe del server handshake sulla porta 80, del ricevitore sulla 3333 e dell'interfaccia web sulla 8080. Si esce dal log con `Ctrl+C`: il servizio continua a funzionare.

Il servizio è già abilitato all'avvio automatico. Per verificarlo, riavvia una volta il Raspberry e controlla che riparta da solo.

Interfaccia web: <http://dmdpi.local:8080/> Digitando l'indirizzo senza porta si viene rediretti automaticamente.

5. Le tre porte

Porta	Uso	Servita da
80	handshake ZeDMD (cablata nel client, non modificabile)	server HTTP dedicato
3333	frame DMD, TCP e UDP	ricevitore ZeDMD
8080	interfaccia web	Flask

L'handshake **non** passa da Flask di proposito: il client di libzedmd legge la risposta con una sola `recv()` e si ferma appena riceve meno di 1024 byte. Se header e corpo arrivano in due pacchetti distinti — come fa Flask — il client legge un corpo vuoto, non riconosce il trasporto TCP e ripiega su UDP. Il server dedicato invia tutto in una sola scrittura.

6. Collegamento a Batocera

Sulla macchina Batocera, via SSH come `root`, crea il file di configurazione di `dmdserver`:

```
cat > /userdata/system/configs/dmdserver/config.ini << 'EOF'
[DMDServer]
AltColor = 1

[ZeDMD]
Enabled = 0

[ZeDMD-WiFi]
Enabled = 1
WiFiAddr = 192.168.0.XXX
EOF
```

Sostituisci `192.168.0.XXX` con l'indirizzo IP del Raspberry, che trovi nella pagina **Impostazioni** dell'interfaccia web.

`[ZeDMD] Enabled = 0` disattiva la ricerca del dispositivo su porta seriale: con il collegamento di rete attivo non serve ed eviterebbe conflitti.

Poi, nel menu di Batocera, attiva il servizio **DMD reale** (non “DMD Web”, che è il simulatore su browser).

Verifica

Con `journalctl -u dmd -f` aperto sul Raspberry, riavvia il servizio DMD di Batocera. Deve comparire:

```
[zedmd] client connesso via TCP: ('192.168.0.XXX', ...)
```

Se invece vedi ripetutamente solo `GET /handshake` senza connessione, il client non sta riuscendo ad aprire lo stream: verifica che `zedmd.http_port` sia 80 e `web.port` sia 8080.

7. Aggiornamento

Ci sono due modi. Il primo non richiede la riga di comando.

7.1 Dall'interfaccia web (consigliato)

Nella pagina **Impostazioni**, sezione *Aggiornamento*, il sistema confronta la versione installata con quella pubblicata su GitHub. Quando ce n'è una nuova compare il pulsante di installazione.

L'aggiornamento è progettato per non poter lasciare il sistema rotto:

1. scarica l'archivio del ramo in una cartella temporanea
2. verifica che ci siano tutti i file attesi e che tutto il Python compili
3. salva una copia dell'installazione corrente in `/var/lib/dmd/backup`
4. sostituisce i file e riavvia il servizio
5. interroga la web UI per capire se il servizio è davvero ripartito
6. se non risponde, ripristina la copia e riavvia di nuovo

L'esito si legge nel riquadro del registro, in fondo alla stessa pagina.

7.2 Da riga di comando

```
cd ~/dmd
git pull
sudo ./update.sh
```

Lo script copia i file aggiornati in `/opt/dmd`, adegua automaticamente la configurazione esistente senza sovrascrivere le tue impostazioni, e riavvia il servizio.

Riavvia sempre Batocera dopo un aggiornamento. Il client ZeDMD mantiene in memoria lo stato della connessione e la contabilità interna delle zone dell'immagine già inviate. Dopo il riavvio del servizio sul Raspberry quello stato non è più valido: senza un riavvio di Batocera il display può restare fermo sull'ultimo contenuto o aggiornarsi solo parzialmente.

L'aggiornamento **non** richiede di ricompilare i binding Python: quel passo si esegue una volta sola in fase di installazione.

8. Interfaccia web

Lingua

L'interfaccia è disponibile in **italiano e inglese**. Alla prima apertura la lingua viene dedotta da quella del browser; il selettore **IT / EN** in alto a destra la cambia in qualsiasi momento e la scelta viene salvata in `/etc/dmd/config.json`. Nella pagina Impostazioni, riportando la voce *Lingua dell'interfaccia* su **predefinito**, si torna a seguire il browser — utile se al DMD accedono persone diverse dai propri dispositivi.

I nomi dei giorni che appaiono **sul pannello** hanno un'impostazione a parte, nella pagina Orologio, e comprendono anche il francese: chi guarda il cabinato non è necessariamente chi configura il sistema.

Il piede di ogni pagina riporta il link al progetto su GitHub.

Le pagine

Impostazioni — luminosità con applicazione immediata, lingua dell'interfaccia, server NTP, fuso orario, ora legale automatica o scostamento UTC manuale, Night mode e Sleep mode, regolazioni fini del driver S-PWM, aggiornamento via rete, indirizzo IP locale e riepilogo delle porte.

Orologio — colori indipendenti per ora e data, formato 12 o 24 ore, lingua dei nomi dei giorni (italiano, francese, inglese), lampeggio dei due punti.

Media — libreria, caricamento file, rilettura della libreria, anteprima immediata, durate e intervalli, adattamento al pannello, modalità pixel art.

Radar — coordinate e raggio, provider ADS-B, scelta dei parametri di volo da mostrare, registro CSV dei passaggi con scaricamento, prova diagnostica di una rotta.

Servizi — attivazione dei quattro servizi, sorgente attualmente a schermo e possibilità di forzarne una invece di lasciar decidere l'arbitro.

Salvare la configurazione

In fondo alla pagina Impostazioni, il riquadro **Configurazione** esporta un file JSON con tutto: taratura del pannello, colori, fasce orarie, servizi, impostazioni del radar. **Fallo dopo ogni modifica importante e tieni il file altrove.**

La ragione è concreta: il codice è su GitHub e si scarica in un minuto, ma la taratura del pannello si trova per tentativi e vive solo su questo Raspberry. Se la scheda SD si guasta — e le schede SD si guastano — quel file è la differenza tra venti minuti e mezza giornata.

La casella *Includi le coordinate del radar* si può togliere quando il file serve per una segnalazione o per qualcun altro: in quel caso la posizione viene esportata a zero.

L'importazione accetta anche file salvati da versioni precedenti, che vengono adeguati automaticamente. La configurazione in uso viene copiata in `/var/lib/dmd/` prima di essere sostituita, e il servizio si riavvia.

9. Come viene deciso cosa appare sul display

Un solo processo può pilotare i GPIO, quindi tutte le sorgenti vivono nello stesso servizio e un arbitro sceglie chi vince:

Priorità	Sorgente
100	ZeDMD
60	Air Radar
50	Media Player
10	Orologio

ZeDMD prende il controllo **immediatamente** appena un client si connette o arriva un frame, e lo mantiene per `grace_seconds` dopo l'ultimo segnale. Il tempo di grazia serve a evitare che l'orologio si intrometta durante le pause normali di Batocera: menu, caricamenti, cambi di schermata.

Sleep mode e Night mode stanno sopra a tutto: Sleep spegne il display, Night ne riduce la luminosità, e Sleep ha la precedenza su Night.

Se Batocera viene spento bruscamente, la connessione non viene chiusa in modo pulito: il ricevitore se ne accorge dopo `client_timeout` secondi di silenzio, poi decorre il tempo di grazia. Con i valori di default il display torna all'orologio entro circa 70 secondi.

10. Comandi utili

```
sudo systemctl start dmd      # avvia
sudo systemctl stop dmd      # ferma
sudo systemctl restart dmd    # riavvia
systemctl status dmd         # stato
journalctl -u dmd -f         # log in tempo reale
journalctl -u dmd -n 100     # ultime 100 righe
curl http://localhost/handshake # verifica dell'handshake
cd ~/dmd && git pull          # scarica l'ultima versione
```

11. Righe bianche, rallentamenti e blocchi

Questa sezione merita attenzione: gli stessi sintomi hanno due cause diverse, che vanno distinte prima di cambiare hardware.

11.1 Righe bianche orizzontali

La libreria genera il segnale del pannello **da programma**, temporizzando i GPIO con precisione di microsecondi. Ogni interruzione del processo — un altro programma che gira, un accesso alla scheda SD, il traffico di rete — allunga il tempo di accensione di una riga, e quella riga appare più chiara. Sono quindi un indicatore di carico, non un guasto del pannello.

Rimedi, in ordine di efficacia:

Intervento	Dove
<code>isolcpus=3</code> — riserva un core al pannello	<code>/boot/firmware/cmdline.txt</code>
<code>dtparam=audio=off</code> — l'audio integrato usa lo stesso hardware PWM	<code>/boot/firmware/config.txt</code>
<code>panel slowdown</code> corretto per il modello	<code>/etc/dmd/config.json</code>
<code>panel.limit_refresh</code> a 60 — un refresh più alto costa CPU senza guadagno visibile	pagina Impostazioni
<code>panel.pwm_bits</code> più basso (8 invece di 11) — meno sfumature, molto meno lavoro	pagina Impostazioni
tenere la libreria media su chiavetta USB anziché su SD	vedi 12.2

11.2 Blocchi, `Bus error`, `Input/output error`

Messaggi come questi **non** sono un problema di velocità:

```
Bus error
-bash: /usr/bin/nice: Input/output error
-bash: /usr/bin/rm: Input/output error
cat: command not found
```

Il sistema non riesce più a **leggere i propri programmi dal disco**. Un Raspberry lento è lento, non perde l'accesso a `/usr/bin`. Le cause possibili sono due, entrambe da verificare prima di comprare una scheda nuova.

Alimentazione insufficiente. Se Raspberry e pannelli condividono lo stesso alimentatore, l'assorbimento dei LED fa scendere la tensione sul Raspberry. Sotto i 4,65 V il controller della scheda SD si resetta e le letture falliscono. È coerente con il fatto che i problemi peggiorano sotto carico.

```
vcgencmd get_throttled          # 0x0 = tutto bene
dmesg | grep -i -E "under-voltage|voltage"
```

Il bit 0 acceso significa sottotensione in corso, il bit 16 che si è verificata almeno una volta dall'accensione. Rimedio: alimentare il Raspberry con un proprio alimentatore, separato da quello dei pannelli, con la massa in comune.

Scheda SD in esaurimento. Scrivere decine di migliaia di file piccoli è tra le cose più dure per una scheda SD. Quando i blocchi iniziano a cedere, i programmi di sistema diventano illeggibili.

```
dmesg | grep -i -E "mmc|ext4|i/o error"
df -h /                          # spazio
df -i /                          # inode
sudo touch /forcefsck && sudo reboot
```

Righe `mmcblk0: error -110 o EXT4-fs error` confermano la diagnosi. Rimedio: scheda nuova di tipo *high endurance* (SanDisk High Endurance o Max Endurance, Samsung PRO Endurance) e reinstallazione: vedi la sezione 12, che spiega come non ritrovarsi nella stessa situazione.

Passare a un Raspberry Pi 4 **non risolve** questo secondo problema: la scheda SD resta la stessa. Risolve invece il primo, perché ha più CPU e perché può avviarsi da SSD USB 3.0, togliendo del tutto la SD dal percorso.

11.3 Librerie media molto grandi

Le raccolte Pixelcade complete arrivano a decine di migliaia di file. Due accorgimenti:

Tieni la libreria fuori dalla scheda SD. Una chiavetta USB montata su `/srv/dmd/media` toglie sia le scritture sia le letture dalla SD:

```
lsblk                          # individua la chiavetta, es. sda1
sudo blkid /dev/sda1           # annota lo UUID
sudo mkdir -p /srv/dmd/media
echo 'UUID=xxxx-xxxx /srv/dmd/media exfat defaults,nofail,uid=1000,gid=1000 0 0' | sudo tee -a /etc/fstab
sudo mount -a
```

Scomatta con priorità bassa. Se devi comunque scrivere sulla SD, limita l'impatto sia sulla CPU sia sul disco:

```
sudo systemctl stop dmd
nice -n 19 ionice -c3 tar xzf pixelcade.tar.gz -C /srv/dmd/media
sync
sudo systemctl start dmd
```

Fermare il servizio durante l'operazione è il singolo accorgimento più utile: il pannello non viene disturbato e l'estrazione ha la macchina per sé.

Dalla versione 1.6 l'elenco dei file viene tenuto in memoria per cinque minuti invece di essere riletto a ogni contenuto e a ogni richiesta della web UI. Con 40.000 file quella scansione continua era essa stessa una causa di righe bianche. Dopo aver copiato file dalla condivisione di rete, usa il pulsante **Rileggi la libreria** nella pagina Media.

12. Installazione resistente all'usura

Questa sezione nasce da un guasto reale: una scheda SD che ha cominciato ad accettare scritture e a restituire dati diversi. Nessun comando segnalava niente — `scp`, `tar` e `cp` riportavano successo — e i file arrivavano della lunghezza giusta con byte nulli al centro. Il servizio non partiva, e l'unico indizio era un `ValueError` sul primo `import`.

Le quattro contromisure qui sotto, messe in opera durante l'installazione, costano venti minuti e cambiano l'ordine di grandezza della vita della scheda.

12.1 Scegliere la scheda giusta

Le sigle grandi sulla confezione — V10, V30, A1, “100 MB/s” — misurano la **velocità**, non la durata. Nessuna dice quanti dati la scheda può scrivere prima di degradarsi, che è il parametro rilevante per un sistema acceso sempre.

Servono schede della linea **high endurance** (SanDisk High Endurance o Max Endurance, Samsung PRO Endurance), nate per dashcam e videosorveglianza, cioè per lo stesso profilo d’uso. Bastano 32 GB: il sistema ne occupa meno di otto e i media stanno altrove.

Evita le schede a marchio concesso in licenza, dove il produttore reale del chip non è dichiarato e cambia tra una partita e l’altra.

Le partizioni non proteggono dall’usura. Dedicare una partizione ai media sembra isolarli, ma il *wear leveling* lavora sull’intero chip: le partizioni esistono solo nello spazio degli indirizzi logici, mentre il controller distribuisce le scritture su tutte le celle fisiche. Una partizione separata aiuta il recupero — reinstalli il sistema e i media restano — ma non allunga la vita della scheda di un giorno. A separare davvero è solo un **supporto fisico diverso**.

12.2 Media su chiavetta USB

È l’intervento con il rapporto migliore fra sforzo e risultato: toglie dalla scheda sia le scritture massive sia le letture continue, e le lascia il solo sistema operativo, che scrive pochissimo.

```
lsblk
```

Individua la chiavetta (di solito `sda1`) e annota lo UUID:

```
sudo blkid /dev/sda1
```

```
sudo mkdir -p /srv/dmd/media
```

Aggiungi la riga a `/etc/fstab` sostituendo lo UUID e il tipo di filesystem (`exfat` per una chiavetta formattata su Windows o Mac, `ext4` se l’hai formattata su Linux):

```
echo 'UUID=xxxx-xxxx /srv/dmd/media exfat defaults,nofail,uid=1000,gid=1000 0 0' | sudo tee -a /etc/fstab
```

```
sudo mount -a && df -h /srv/dmd/media
```

L’opzione `nofail` è importante: senza, se un giorno la chiavetta non c’è il Raspberry si ferma all’avvio invece di proseguire.

12.3 Radice in sola lettura

Raspberry Pi OS può montare la radice in sola lettura, con tutte le scritture dirottate in memoria e scartate al riavvio. Su un sistema che una volta configurato non cambia più, è la misura che allunga di più la vita della scheda: le scritture scendono praticamente a zero.

```
sudo raspi-config
```

Performance Options → Overlay File System → abilita l’overlay e imposta la partizione di avvio in sola lettura. Poi riavvia.

Il prezzo da pagare è che da quel momento nessuna modifica sopravvive al riavvio: né gli aggiornamenti del software, né le impostazioni cambiate dalla web UI, né la configurazione. Per intervenire si disattiva, si lavora, si riattiva:

```
sudo raspi-config nonint disable_overlayfs && sudo reboot
```

```
cd ~/dmd && git pull && sudo ./update.sh
```

```
sudo raspi-config nonint enable_overlayfs && sudo reboot
```

Se ti dimentichi di riattivarlo non succede niente di grave: il sistema funziona esattamente come prima, semplicemente senza la protezione.

Attivalo **alla fine**, quando pannello, colori e servizi sono a posto. Con l’overlay attivo la web UI accetta le modifiche e le mostra, ma al riavvio tutto torna com’era — un comportamento che fa perdere un pomeriggio a chiunque non se lo ricordi.

12.4 Esportare la configurazione

Dalla pagina Impostazioni, riquadro **Configurazione**. È l’unica parte del sistema che non si può riscaricare: il codice sta su GitHub, la taratura del pannello sta solo qui.

Esportala **dopo ogni modifica importante** e tieni il file su un’altra macchina. Con quel file, rimettere in piedi tutto su una scheda nuova sono venti minuti; senza, si ricomincia la campagna di prove sul pannello.

13. Risoluzione problemi

Sintomo	Causa probabile	Rimedio
Il servizio non parte, errore su <code>rgbmatrix</code>	binding Python non compilati	rilanciare <code>install.sh</code>
Pannello nero ma servizio attivo	nessuna sorgente abilitata	attivare Orologio dalla pagina Servizi
<code>Address already in use</code> sulla porta 80	un altro web server è attivo	fermarlo (<code>lighttpd</code> , <code>apache2</code> , <code>nginx</code>)
Handshake ripetuto, nessun frame	porte scambiate	<code>zedmd.http_port = 80</code> , <code>web.port = 8080</code>
Righe bianche orizzontali	carico della CPU o del disco	sezione 11.1
<code>Bus error</code> , <code>Input/output error</code>	alimentazione o scheda SD	sezione 11.2
L'orologio interrompe Batocera	tempo di grazia troppo corto	alzare <code>zedmd.grace_seconds</code>
Dopo un aggiornamento il display resta fermo	stato del client non più valido	riavviare Batocera
I file copiati via SMB non compaiono	elenco ancora in cache	pulsante <i>Rileggi la libreria</i>
Il radar non mostra nulla	coordinate a 0	inserirle nella pagina Radar
La rotta non appare	volo non presente nel servizio rotte	usare la prova diagnostica nella pagina Radar

14. Struttura dei file installati

<code>/opt/dmd/dmdd.py</code>	servizio principale, arbitro e ciclo di rendering
<code>/opt/dmd/display.py</code>	proprietario esclusivo del pannello
<code>/opt/dmd/dmdconf.py</code>	configurazione persistente
<code>/opt/dmd/webui.py</code>	interfaccia web (Flask)
<code>/opt/dmd/ota.py</code>	aggiornamento via rete da GitHub
<code>/opt/dmd/zedmd_http.py</code>	server dell'handshake ZeDMD sulla porta 80
<code>/opt/dmd/sources/</code>	sorgenti di contenuto
<code>/etc/dmd/config.json</code>	configurazione
<code>/var/lib/dmd/backup/</code>	copia dell'installazione precedente
<code>/var/lib/dmd/flights.csv</code>	registro dei passaggi aerei
<code>/var/lib/dmd/ota.log</code>	registro degli aggiornamenti
<code>/srv/dmd/media/</code>	libreria media, condivisa via SMB
<code>/etc/systemd/system/dmd.service</code>	